

Práctica Global: Probando nuestro entorno de Cluster Real con Varias Posibilidades

Objetivo: Comprender y ejecutar trabajos MapReduce en un entorno de clúster real utilizando diferentes interfaces: la API nativa de Java, Hadoop Streaming con Shell Scripting y Hadoop Streaming con Python.

Requisitos previos:

- Tener el fichero cite75_99.txt (datos de patentes de EE.UU.).
- Subir el fichero a HDFS en la ruta /practicass.

+1

Bash

```
hdfs dfs -put cite75_99.txt /practicass
```

Parte 1: El Enfoque Clásico (Java MapReduce)

En esta sección, crearemos un programa Java robusto para agrupar las patentes por su código de citación principal.

1. Crear el código fuente

Crea un fichero llamado MyJob.java con el siguiente código. Este código define un Mapper que separa las citas por comas y un Reducer que concatena los valores.

+3

Java

```
import java.io.IOException;

import java.util.Iterator;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
```

```

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class MyJob extends Configured implements Tool {

    // MAPPER: Separa la línea por comas e invierte el orden (key, value)
    public static class MapClass extends Mapper<LongWritable, Text, Text, Text> {

        public void map(LongWritable key, Text value, Context context) throws IOException,
        InterruptedException {

            String[] citation = value.toString().split(",");

            if (citation.length > 1) {

                context.write(new Text(citation[1]), new Text(citation[0]));

            }

        }

    }

    // REDUCER: Concatena todos los valores asociados a una misma clave
    public static class Reduce extends Reducer<Text, Text, Text, Text> {

        public void reduce(Text key, Iterable<Text> values, Context context) throws IOException,
        InterruptedException {

            String csv = "";

            for (Text val : values) {

                if (csv.length() > 0) csv += ",";

                csv += val.toString();

            }

            context.write(key, new Text(csv));

        }

    }

    // DRIVER: Configuración del trabajo

```

```

public int run(String[] args) throws Exception {
    Configuration conf = getConf();
    Job job = new Job(conf, "MyJob");
    job.setJarByClass(MyJob.class);

    FileInputFormat.setInputPaths(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    job.setMapperClass(MapClass.class);
    job.setReducerClass(Reduce.class);

    job.setInputFormatClass(TextInputFormat.class);
    job.setOutputFormatClass(TextOutputFormat.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);

    return job.waitForCompletion(true) ? 0 : 1;
}

```

```

public static void main(String[] args) throws Exception {
    int res = ToolRunner.run(new Configuration(), new MyJob(), args);
    System.exit(res);
}

```

2. Compilar y Empaquetar

Para ejecutarlo en el clúster, necesitamos compilar el código y crear un archivo .jar.

1. **Exportar variables de entorno** para incluir las librerías de Hadoop:

Bash

```
export HADOOP_CLASSPATH=$JAVA_HOME/lib/tools.jar
```

2. **Compilar:**

Bash

```
hadoop com.sun.tools.javac.Main MyJob.java
```

3. Crear el JAR:

Bash

```
jar cvf MyJob.jar MyJob*.class
```

3. Ejecutar y Verificar

Lanzamos el trabajo especificando el archivo de entrada y el directorio de salida (ej. /resultado7):

Bash

```
hadoop jar MyJob.jar MyJob /practicas/cite75_99.txt /resultado7
```

Verifica el resultado:

Bash

```
hdfs dfs -cat /resultado7/part-r-00000 | head
```

Parte 2: Prototipado Rápido (Hadoop Streaming con Shell)

A veces no es necesario escribir Java. Podemos usar comandos de Linux para simular Mappers y Reducers gracias a la librería **Hadoop Streaming**.

Caso A: Extracción y Limpieza

Usaremos cut para extraer el campo 2 (Mapper) y uniq para eliminar duplicados (Reducer).

+1

Nota: Verifica la versión de tu jar de streaming (ej. 2.9.0 o la que tengas instalada).

Bash

```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-*.jar \
```

```
-input /practicas/cite75_99.txt \
```

```
-output /resultado8 \
```

```
-mapper 'cut -f 2 -d ,' \
```

```
-reducer 'uniq'
```

Caso B: Conteo sin Reducer (Map Only)

Si solo queremos contar líneas, podemos prescindir de la fase de reducción explícita estableciendo las tareas de reducción a 0. Esto generará múltiples ficheros de salida correspondientes a los bloques de datos (splits).

+2

Bash

```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-*.jar \  
-D mapred.reduce.tasks=0 \  
-input /practicas/cite75_99.txt \  
-output /resultado9 \  
-mapper 'wc -l'
```

Observación: Al listar el directorio /resultado9, verás dos ficheros (part-00000 y part-00001) porque el fichero de entrada ocupa unos 200MB y el tamaño de bloque es de 128MB, generando dos splits.

+1

Parte 3: Scripting Avanzado (Hadoop Streaming con Python)

Para lógica más compleja que un comando de shell, pero más rápida de escribir que Java, usamos Python. Haremos un muestreo aleatorio de datos.

1. Crear el Script Python

Crea un archivo rand.py que imprima líneas aleatoriamente basado en un porcentaje.

+1

Python

```
#!/usr/bin/env python
```

```
import sys, random
```

```
# El porcentaje se pasa como argumento
```

```
percent = int(sys.argv[1]) if len(sys.argv) > 1 else 10
```

```
for line in sys.stdin:
```

```
    if (random.randint(1,100) <= percent):
```

```
        print line.strip()
```

2. Ejecutar con Streaming

Debemos usar el parámetro -file para enviar el script de Python a todos los nodos del clúster.

Bash

```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-*.jar \  
-D mapred.job.reducers=1 \  
-input /practicas/cite75_99.txt \  
-file /resultado9/rand.py
```

-output /resultado11 \
-mapper 'python rand.py 10' \
-file rand.py

Resumen de Comprobaciones

Para finalizar la práctica, puedes monitorizar todos estos trabajos:

1. **Vía Web:** Accede a la UI de YARN (puerto 8088 usualmente) para ver el estado SUCCEEDED de tus aplicaciones.

+1

2. **Vía Consola:** Usa `hdfs dfs -ls /` para ver los directorios `/resultado7`, `/resultado8`, `/resultado9` y `/resultado11`.

Parte 4: Optimización y Control de Reducers

En los ejercicios anteriores hemos visto tres comportamientos distintos controlados por el número de reducers. Entender esto te permite decidir si quieres velocidad (paralelismo) o agregación total (un solo resultado).

1. El caso de "Map-Only" (0 Reducers)

En la **Parte 2 - Caso B** (Shell con `wc -l`), usamos este comando: `-D mapred.reduce.tasks=0`

- **¿Qué ocurre?** El trabajo termina justo después de la fase de Map. No hay fase de mezcla (shuffle) ni reducción.
- **Resultado en HDFS:** Obtienes tantos ficheros de salida como bloques de datos o "splits" tenga tu fichero de entrada.
 - En el PDF se ve que el fichero `cite75_99.txt` ocupaba unos 200MB. Como el tamaño de bloque por defecto suele ser 128MB, se generan **2 splits**.
 - Por eso, al listar el resultado, aparecían `part-00000` y `part-00001`.

+1

- **Uso:** Ideal para filtrado de datos (ETL) donde no necesitas agrupar, solo transformar línea a línea. Es la forma más rápida de procesar.

2. Agregación Total (1 Reducer)

En la **Parte 3** (Python con `rand.py`), usamos: `-D mapred.job.reducers=1`

- **¿Qué ocurre?** Obligamos a que todos los datos procesados por los Mappers viajen a través de la red hacia **un único nodo** para ser procesados por una sola tarea Reducer.
- **Resultado en HDFS:** Un único fichero (`part-00000`).

- **Peligro:** Si procesas Terabytes de datos, este único Reducer se convertirá en un cuello de botella brutal (el famoso "curse of the last reducer").
- **Uso:** Necesario cuando requieres un orden global total o un único archivo de resumen final, pero debe usarse con cuidado en Big Data.

3. El término "Deprecado" (Avisos del Clúster)

Si te fijas en los logs que aparecen en el PDF de la práctica de Shell, el clúster nos lanza una advertencia importante:

INFO Configuration.deprecation: mapred.reduce.tasks is deprecated. Instead, use mapreduce.job.reduces

Esto es muy común en entornos reales. Los tutoriales antiguos usan mapred.* (API vieja), pero en clústers modernos (YARN/Hadoop 2.x o 3.x) deberíamos empezar a usar las propiedades nuevas.

- **Para la práctica:** Sigue usando -D mapred.reduce.tasks si así funciona tu script, pero toma nota mental de que en producción moderna deberías usar -D mapreduce.job.reduces.

Ejercicio

Para probar esto en tu entorno, puedes repetir el comando de Python o Shell cambiando simplemente ese número:

1. **Lanza el script de Python con 2 Reducers:** Cambia el -D mapred.job.reducers=1 por =2.
 - *Resultado esperado:* Verás dos ficheros en la carpeta de salida. Cada uno contendrá una parte aleatoria de las líneas seleccionadas.